

ASP.NET Core Web API para Backend

Luis Antonio Castillo Javier

Tiempo estimado: 1 hr 30 min

Universidad San Carlos de Guatemala
Facultad de Ingeniería
Escuela de Ingeniería en Ciencias y Sistemas.
Segundo Semestre 2026
[Laboratorio - Introducción a la Programación y Computación 2 - Sección A]

1. Marco Formativo

1.1 Valores

Nombre del valor	¿Cómo se aplica en tu laboratorio?
Integridad	Al desarrollar una Web API REST, la integridad se manifiesta en respetar el contrato de la API: responder con los códigos HTTP correctos, no alterar datos sin la debida autorización y documentar fielmente cada endpoint. Un backend íntegro es predecible y confiable para cualquier cliente que lo consuma.

1.2 Competencia(s)

Tipo de Competencia	
Competencia General	Diseñar e implementar servicios backend mediante APIs REST utilizando ASP.NET Core Web API, aplicando la arquitectura Model-Service-Controller y los métodos HTTP estándar.
Competencia Específica	Construir una Web API REST funcional con ASP.NET Core que exponga operaciones CRUD a través de endpoints HTTP (GET, POST, PUT, DELETE), organizando el código en capas Model, Service y Controller, y verificando las respuestas con herramientas de prueba como Postman.

2. Palabras Clave

Backend: Parte invisible pero esencial de un sistema web, encargada de manejar la lógica y el procesamiento de datos necesarios para que todo funcione correctamente.

API (Application Programming Interface): Conjunto de reglas que permite que dos aplicaciones se comuniquen entre sí de forma controlada y estandarizada.

REST (Representational State Transfer): Conjunto de principios arquitectónicos para diseñar servicios web que permiten comunicación a través del protocolo HTTP de manera simple y escalable.

ASP.NET Core Web API: Framework de Microsoft para crear servicios backend basados en HTTP, diseñado para comunicarse con frontends, apps móviles o sistemas externos usando JSON.

Controller: Clase responsable de recibir solicitudes HTTP y devolver respuestas al cliente; actúa como intermediario entre el cliente y la lógica de negocio.

Model: Clase C# que define la estructura y propiedades de los datos que maneja la aplicación.

Service: Capa donde se implementa la lógica de negocio; separa las reglas de la aplicación del manejo directo de solicitudes HTTP.

Endpoint: URL específica dentro de una API que recibe y responde a solicitudes HTTP asociadas a una operación concreta.

3. Introducción

El desarrollo de aplicaciones web modernas se apoya en una separación clara entre el frontend (lo que el usuario ve) y el backend (la lógica que procesa los datos). Esta lectura aborda el backend desde la perspectiva del ecosistema .NET de Microsoft, utilizando ASP.NET Core Web API como framework principal para construir servicios HTTP robustos, escalables y listos para producción.

A lo largo de este documento se explorarán los conceptos fundamentales que hacen posible la comunicación entre aplicaciones a través de APIs REST: desde qué es el backend y cómo funciona una API, hasta la arquitectura interna de ASP.NET Core con sus capas de Model, Service y Controller. El objetivo es que el estudiante comprenda no solo cómo escribir el código, sino también por qué cada componente existe y cuál es su responsabilidad dentro del sistema.

4. Contenidos

4.1 ¿Qué es el Backend?

El backend es la parte invisible pero esencial de una aplicación web. Mientras el frontend se ocupa de la interfaz que el usuario ve e interactúa, el backend gestiona toda la lógica de negocio, el acceso a bases de datos, la autenticación de usuarios y la seguridad del sistema. En términos simples, es el motor que hace que todo funcione detrás de escena.

El backend se ejecuta en un servidor: una aplicación especializada que recibe solicitudes del navegador o de cualquier cliente, las procesa y responde adecuadamente. En el ecosistema .NET, ASP.NET Core actúa como ese servidor, usando Kestrel como servidor HTTP integrado. Para que la comunicación entre cliente y servidor sea estructurada y predecible, se utilizan las APIs.

Las responsabilidades principales del backend incluyen:

- Almacenar y recuperar datos de una base de datos.
- Procesar formularios y validar información de entrada.
- Autenticar usuarios y gestionar sesiones o tokens.
- Exponer funcionalidades a través de endpoints HTTP que el frontend puede consumir.

4.2 ¿Qué es una API REST?

API (Application Programming Interface)

Una API es un conjunto de reglas y definiciones que permite que dos aplicaciones se comuniquen entre sí. En el contexto del desarrollo web, la API actúa como un puente entre el frontend y el backend: el frontend solicita datos o acciones, y la API los provee de forma controlada y segura. Tres características clave definen una API:

- Interfaz definida: expone funciones específicas que otros programas pueden usar sin conocer su implementación interna.
- Contrato de comunicación: define cómo deben estructurarse las solicitudes y respuestas (formato, headers, métodos HTTP).
- Reutilización: permite que múltiples clientes (web, móvil, escritorio) consuman los mismos servicios simultáneamente.

REST (Representational State Transfer)

REST es un estilo arquitectónico para diseñar servicios web. No es un protocolo ni un estándar formal, sino un conjunto de principios que, cuando se aplican correctamente, producen APIs simples, escalables e interoperables. Sus principios fundamentales son:

- Cliente-servidor: separación estricta entre la interfaz de usuario y el almacenamiento de datos. Esto permite que evolucionen de forma independiente.
- Sin estado (Stateless): cada solicitud del cliente debe contener toda la información necesaria para ser procesada. El servidor no guarda contexto entre solicitudes.
- Caché: las respuestas pueden marcarse como cacheables para mejorar el rendimiento y reducir la carga del servidor.
- Interfaz uniforme: uso coherente y predecible de los métodos HTTP (GET, POST, PUT, DELETE, PATCH) para cada tipo de operación.
- Sistema en capas: la arquitectura puede incluir intermediarios como proxies, balanceadores de carga o gateways sin que el cliente lo sepa.

Una API REST combina ambos conceptos: es una API que sigue los principios REST, usando HTTP como protocolo de comunicación y JSON como formato de intercambio de datos. El cliente envía una solicitud con un método HTTP y una URL, y el servidor responde con un código de estado y los datos solicitados.

4.3 Métodos HTTP en una API REST

Los métodos HTTP definen la intención de cada solicitud. Una API REST bien diseñada asigna un método distinto a cada tipo de operación sobre sus recursos:

Tabla 1 — Métodos HTTP y su uso en ASP.NET Core Web API

Método	Descripción	Uso típico	Código de respuesta
GET	Recupera un recurso o lista de recursos sin modificar datos.	Obtener estudiantes, productos, usuarios.	200 OK
POST	Crea un nuevo recurso en el servidor con los datos enviados en el cuerpo.	Registrar un nuevo estudiante o producto.	201 Created
PUT	Reemplaza completamente un recurso existente con los datos proporcionados.	Actualizar todos los campos de un registro.	200 OK

DELETE	Elimina un recurso específico identificado por su ID en la URL.	Eliminar un estudiante por ID.	200 OK / 204
PATCH	Modifica parcialmente un recurso; solo se envían los campos que cambian.	Actualizar únicamente el nombre de un usuario.	200 OK

4.4 ¿Qué es un Framework y por qué usar ASP.NET Core?

Concepto de Framework

Un framework es un conjunto de herramientas, bibliotecas y convenciones que facilitan el desarrollo de software al proveer una estructura base sobre la cual construir aplicaciones. En lugar de escribir desde cero la gestión de rutas, conexiones HTTP o validaciones, el desarrollador aprovecha lo que el framework ya resuelve y se concentra en la lógica de negocio específica de su aplicación.

Un framework web backend típicamente agiliza: el manejo de rutas y endpoints, la conexión a bases de datos, la validación de datos de entrada, la implementación de autenticación, y la generación de respuestas HTTP con el formato correcto.

ASP.NET Core Web API

ASP.NET Core Web API es el framework de Microsoft para crear servicios backend basados en HTTP. Sus ventajas principales son:

- Alto rendimiento: capaz de manejar miles de solicitudes concurrentes sin degradar el rendimiento, gracias al servidor Kestrel y al modelo asíncrono de .NET.
- Multiplataforma: funciona en Windows, Linux y macOS, facilitando el despliegue en cualquier entorno.
- Modular: solo se incluyen los componentes necesarios, produciendo código más limpio, mantenible y fácil de escalar.
- Serialización automática: convierte objetos C# a JSON automáticamente mediante System.Text.Json, sin configuración adicional.
- Integración nativa: se conecta fácilmente con Entity Framework Core (base de datos), Identity (autenticación) y otros paquetes del ecosistema .NET.

4.5 Arquitectura de ASP.NET Core Web API: Model, Service y Controller

Una Web API bien estructurada separa sus responsabilidades en tres capas principales. Esta separación facilita el mantenimiento, las pruebas unitarias y la escalabilidad del sistema.

Model (Modelo)

El Model es la capa más básica: define la estructura de los datos que maneja la aplicación. En ASP.NET Core, un modelo es simplemente una clase C# con propiedades. Por ejemplo, un modelo Producto podría tener las propiedades Id (int), Nombre (string) y Precio (decimal). Los

modelos sirven como contratos de datos: definen qué información se envía y recibe entre las capas del sistema y, por extensión, qué JSON se produce y consume en los endpoints.

Service (Servicio)

El Service es la capa donde reside la lógica de negocio. Aquí se implementan las reglas que gobiernan cómo se procesan, validan y transforman los datos antes de guardarlos o devolverlos. Por ejemplo, un `EstudianteService` puede validar que el carnet no esté duplicado antes de agregar un estudiante. Separar la lógica en un servicio evita que el controlador se vuelva una clase masiva difícil de mantener, y permite reutilizar esa lógica desde distintos controladores si fuera necesario. Los servicios se registran en el contenedor de inyección de dependencias de ASP.NET Core (`Program.cs`) y se proveen automáticamente a los controladores.

Controller (Controlador)

El Controller es la capa de entrada: recibe las solicitudes HTTP, extrae los datos del cuerpo o la URL, delega el trabajo al servicio correspondiente y devuelve la respuesta HTTP apropiada. En ASP.NET Core, los controladores heredan de `ControllerBase` y se decoran con `[ApiController]` y `[Route("api/[controller]")]`. Cada método del controlador se mapea a un endpoint HTTP mediante atributos como `[HttpGet]`, `[HttpPost]`, `[HttpPut]` y `[HttpDelete]`.

Rutas y atributos HTTP

A diferencia de frameworks como Flask (Python), que usan decoradores de función, ASP.NET Core usa atributos en C# para definir las rutas y los métodos HTTP de cada endpoint. La ruta base del controlador se define con `[Route("api/[controller]")]`, donde `[controller]` se reemplaza automáticamente por el nombre de la clase sin el sufijo "Controller". Los métodos se anotan con `[HttpGet]`, `[HttpPost("ruta")]`, `[HttpGet("{id}")]`, etc.

4.6 Caso de Estudio: API REST de Estudiantes

Para ilustrar todos los conceptos anteriores de manera integrada, se desarrolla una API REST completa que gestiona una lista de estudiantes en memoria. El sistema implementa las cuatro operaciones CRUD básicas.

Definición del Model

Se crea la clase `Estudiante` en la carpeta `Models` con tres propiedades: `Id` (int), `Nombre` (string) y `Carnet` (string). Este modelo define la estructura JSON que la API producirá y consumirá. Cuando ASP.NET Core serialice un objeto `Estudiante`, el resultado será: `{ "id": 1, "nombre": "Ana García", "carnet": "202101234" }`.

Implementación del Service

El `EstudianteService` almacena los estudiantes en una `List<Estudiante>` en memoria. Expone cinco métodos: `ObtenerTodos()` retorna la lista completa; `ObtenerPorId(int id)` busca por ID; `Agregar(Estudiante e)` añade un nuevo estudiante asignándole un ID autoincremental;

Actualizar(Estudiente e) reemplaza el objeto existente; y Eliminar(int id) lo remueve de la lista. El servicio se registra como Singleton en Program.cs (builder.Services.AddSingleton<EstudianteService>()) para que la misma instancia, y por tanto la misma lista en memoria, persista durante toda la vida de la aplicación.

Implementación del Controller

El EstudiantesController hereda de ControllerBase y usa la ruta base api/estudiantes. Sus cuatro métodos son:

- [HttpGet] → GetAll(): llama a ObtenerTodos() y retorna Ok(lista) con código 200.
- [HttpGet("{id}")] → GetById(int id): si el estudiante existe retorna Ok(estudiante); si no, retorna NotFound() con código 404.
- [HttpPost] → Create([FromBody] Estudiante e): llama a Agregar(e) y retorna CreatedAtAction(...) con código 201.
- [HttpDelete("{id}")] → Delete(int id): si el estudiante existe lo elimina y retorna NoContent() (204); si no, retorna NotFound().

Tabla 2 — Resumen de endpoints de la API de Estudiantes

Método	URL	Acción	Respuesta
GET	api/estudiantes	Obtener todos los estudiantes	200 OK + JSON array
GET	api/estudiantes/{id}	Obtener un estudiante por ID	200 OK / 404
POST	api/estudiantes	Crear un nuevo estudiante	201 Created + JSON
PUT	api/estudiantes/{id}	Actualizar todos los datos del registro	200 OK / 404
DELETE	api/estudiantes/{id}	Eliminar un estudiante por ID	204 No Content / 404

Verificación con Postman

Una vez que la API está corriendo (dotnet run), se verifica cada endpoint desde Postman. Para el GET a api/estudiantes se espera un arreglo JSON vacío [] si no hay datos, o con objetos si ya se crearon. Para el POST se envía en el cuerpo (body → raw → JSON) un objeto con nombre y carnet, y se espera el código 201 con el objeto creado incluyendo su ID asignado. Un código diferente al esperado (por ejemplo 400 Bad Request) indica un error en el formato del JSON enviado o en la definición del modelo.

5. Conclusiones

ASP.NET Core Web API representa una solución madura, de alto rendimiento y multiplataforma para construir servicios backend en el ecosistema .NET. La separación en capas

Model-Service-Controller no solo organiza el código de forma lógica, sino que también facilita las pruebas, el mantenimiento y la colaboración en equipos de desarrollo. Comprender los principios REST y el uso correcto de los métodos HTTP es fundamental para diseñar APIs que sean intuitivas, predecibles y fáciles de consumir desde cualquier cliente, ya sea un frontend web, una aplicación móvil o un sistema externo.

El conocimiento adquirido en esta lectura constituye la base sobre la que se construirán los temas siguientes del curso: la integración con bases de datos mediante Entity Framework Core, la autenticación con JWT, y el consumo de la API desde un frontend desarrollado con Razor Pages o tecnologías JavaScript modernas.

6. Referencias

1. Microsoft. (2024). Tutorial: Create a web API with ASP.NET Core. Microsoft Learn. <https://learn.microsoft.com/en-us/aspnet/core/tutorials/first-web-api>
2. Microsoft. (2024). ASP.NET Core Web API documentation. Microsoft Learn. <https://learn.microsoft.com/en-us/aspnet/core/web-api>
3. Platzi. (2018, febrero 21). Qué es Frontend y Backend: principal diferencia y ejemplos. <https://platzi.com/blog/que-es-frontend-y-backend/>
4. Lucena, P. (2023, mayo 6). ¿Qué es el framework? CESUMA – Maestrías y MBA. <https://www.cesuma.mx/blog/que-es-el-framework.html>
5. MDN Web Docs. (2024). HTTP request methods. Mozilla Developer Network. <https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods>
6. Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures (Doctoral dissertation). University of California, Irvine.